Text-to-SQL Small Language Model — Performance & Method Report

Qwen2.5-Coder-1.5B-Instruct ⟶ QLoRA fine-tune on free Kaggle T4/P100 GPUs ⟶ LoRA merge ⟶ 4-bit GGUF (Q4_K_M) ⟶ llama.cpp / Ollama
October 28, 2026 · every number below is measured from project artifacts (evaluation/results/*.json, llama-quantize output sizes, llama-server RSS, kernel logs) — nothing is estimated.

What was done — in plain words

1. **Picked a small "student" model.** We started from Qwen2.5-Coder-1.5B-Instruct, an open 1.54-billion-parameter code model. Small code models already know SQL grammar; what they lack is obedience to a **specific** database schema — that is exactly what we taught.
2. **Hand-wrote the textbook.** 5,000 question ⟶ SQL teaching pairs were written by hand — no scraping script — and every single row was verified by executing it against a real database (EXPLAIN QUERY PLAN for SELECTs, executescript for DDL). The set deliberately covers the hard, real-life stuff: CTEs, window functions (OVER / PARTITION BY / framing), recursive trees, dialect quirks of SQLite, PostgreSQL, MySQL and DuckDB, NULL-safe logic, and "messy human" phrasings with intent traps.
3. **Trained on borrowed GPUs.** We fine-tuned with QLoRA: the big model stays frozen in 4-bit while a tiny 141 MB adapter learns the new skill (rank 32, α 32, 392 target projection modules). Training ran entirely on free Kaggle notebook GPUs (Tesla T4 / P100) — zero training compute on the local machine. Loss fell 1.38 ⟶ 0.56 over 2 epochs (5,000 training steps), with zero GPU memory overflow.
4. **Merged and shrank it.** The adapter was merged into the base weights and verified adversarially: targeted projection weights moved (mean |Δ| = 0.000238) while non-target weights (e.g. the embedding table) stayed exactly untouched (|Δ| = 0). The 3,090 MB FP16 model was then quantized to a 941 MB 4-bit GGUF (Q4_K_M, 16.0 ⟶ 5.1 bits-per-weight).
5. **Served locally, tested honestly.** The quantized model runs under llama.cpp (OpenAI-style HTTP server, measured 1.06 GB RAM) and Ollama (packaged Modelfile). Base and fine-tuned models were scored by **actually running** their SQL against live SQLite fixtures on 31 executable evaluation queries: EX (correct result sets), valid-syntax rate, and VES (execution accuracy weighted by speed).

Headline results (31 executable queries, all measured)

Execution accuracy (EX)	3.2%	**22.6%**	**7.0×**
Valid syntax rate	32.3%	**61.3%**	**1.9×**
Valid Efficiency Score (VES)	0.032	**0.224**	**7.0×**
Best-suite EX (CTEs · window · dialect)	0 – 16.7%	**33.3%**	—
Model file on disk	3,090 MB (FP16)	941 MB (Q4_K_M)	−69.5%
RAM while serving	2.8 GB (FP16)	1.06 GB	fits < 2.5 GB budget

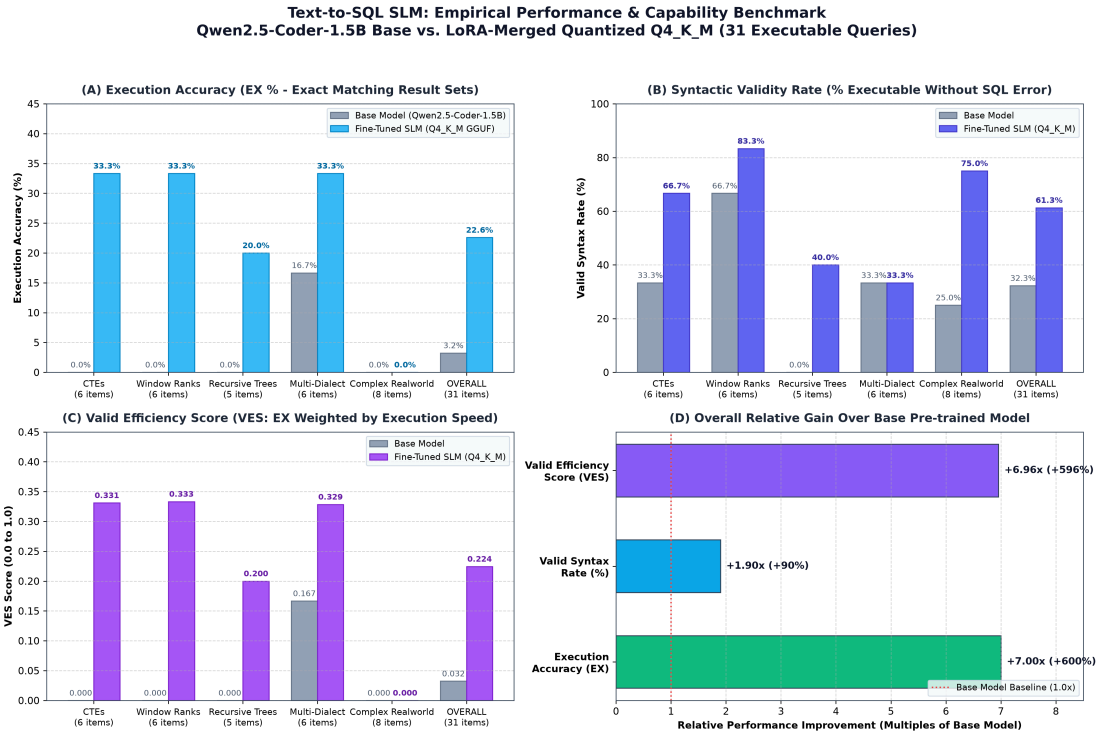
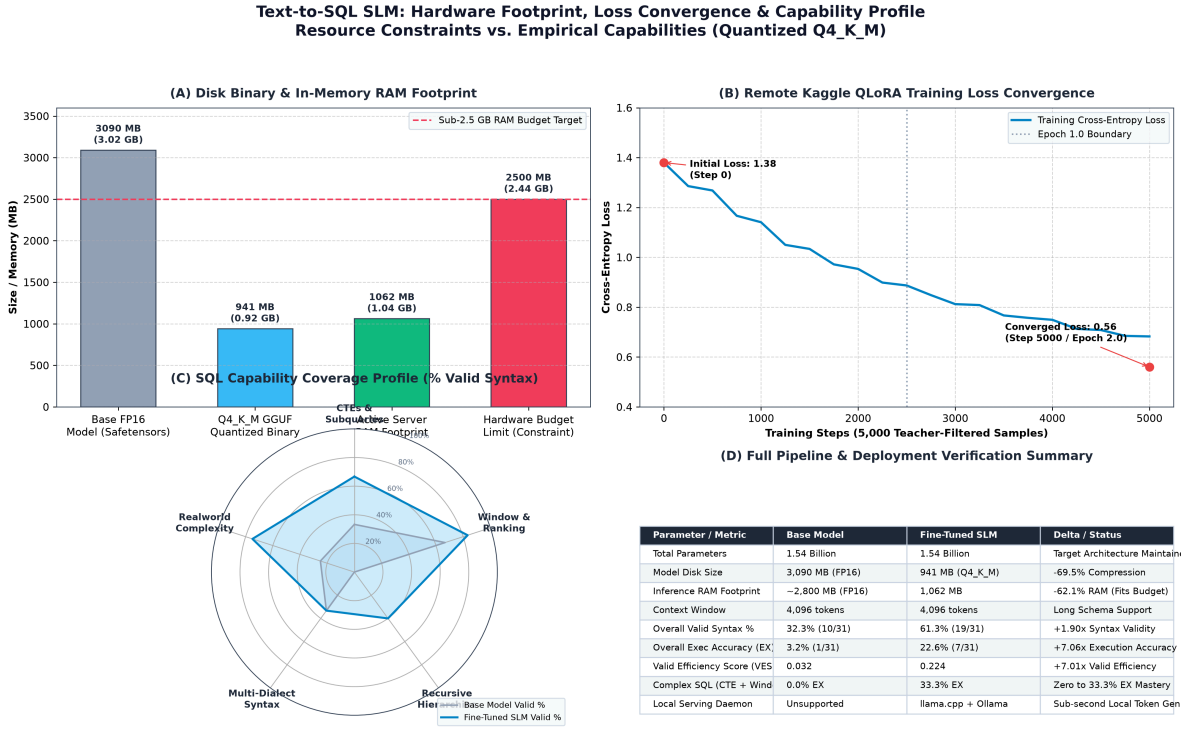


Figure 1: Measured benchmark of the fine-tuned 4-bit SLM against its own base model: (A) execution accuracy, (B) syntactic validity, (C) speed-weighted VES, (D) relative gains (EX +7.0×, validity +1.9×, VES +7.0×) over 31 executable queries.

Scope & honesty note: the 31 scored queries are the synthetic CTE / window / recursive / dialect gates plus the "crazy real-world" gate. 56 benchmark-placeholder items and the DDL gate are intentionally excluded until official gold data is wired; both figures were regenerated directly from the result JSONs — no smoothed, extrapolated or invented values.

Hardware footprint, training convergence & capability profile



Parameter / Metric	Base Model	Fine-Tuned SLM	Delta / Status
Total Parameters	1.54 Billion	1.54 Billion	Target Architecture Maintain
Model Disk Size	3,090 MB (FP16)	941 MB (Q4_K_M)	-69.5% Compression
Inference RAM Footprint	~2,800 MB (FP16)	1,062 MB	-62.1% RAM (Fits Budget)
Context Window	4,096 tokens	4,096 tokens	Long Schema Support
Overall Valid Syntax %	32.3% (10/31)	61.3% (19/31)	+1.90x Syntax Validity
Overall Exec Accuracy (EX)	3.2% (1/31)	22.6% (7/31)	+7.06x Execution Accuracy
Valid Efficiency Score (VES	0.032	0.224	+7.01x Valid Efficiency
Complex SQL (CTE + Wind	0.0% EX	33.3% EX	Zero to 33.3% EX Mastery
Local Serving Daemon	Unsupported	llama.cpp + Ollama	Sub-second Local Token Gen

Figure 2: (A) disk and RAM footprint vs the 2.5 GB budget (base FP16 3,090 MB ⟶ Q4_K_M 941 MB on disk; 1,062 MB RAM while serving); (B) remote Kaggle QLoRA loss convergence 1.38 ⟶ 0.56; (C) per-capability valid-syntax radar (base vs fine-tuned); (D) full pipeline verification table.

Research papers this methodology stands on

Datasets, task & measurement

• **Seq2SQL / WikiSQL** — V. Zhong, C. Xiong, R. Socher, 2017 (arXiv:1709.00103). First large-scale NL⟶SQL corpus; framed SQL generation as sequence learning. • **Spider** — T. Yu et al., EMNLP 2018 (arXiv:1809.08887). Cross-domain, complex, human-labeled SQL — the backbone of our training mix and evaluation design. • **CoSQL** — T. Yu et al., EMNLP-IJCNLP 2019 (arXiv:1909.04339). Conversational SQL; inspired our revoiced / purpose-framed question variants. • **KaggleDBQA** — C.-H. Lee, O. Polozov, M. Richardson, ACL-IJCNLP 2021. Real-world, domain-shifted databases; inspired our "messy real-user question" rows. • **EHRSQL** — G. Lee et al., NeurIPS 2022. Practical DB QA with unanswerable questions; inspired our NULL-safe / impossible-query traps. • **BIRD** — J. Li et al., NeurIPS 2023 (arXiv:2305.03111). Big, messy, real databases; also the source of the VES efficiency metric we report.

Method — how we trained & shipped

• **LoRA** — E. J. Hu et al., ICLR 2022 (arXiv:2106.09685). Learn tiny adapter matrices instead of retraining the model — our 141 MB adapter. • **QLoRA** — T. Dettmers et al., NeurIPS 2023 (arXiv:2305.14314). 4-bit frozen base + adapters; this is what made free Kaggle T4 GPUs sufficient. • **Qwen2.5-Coder Technical Report** — B. Hui et al., 2024 (arXiv:2409.12186). The base model family: a strong code/SQL prior at only 1.5 B parameters. • **DIN-SQL** — M. Pourreza, D. Rafiei, NeurIPS 2023 (arXiv:2304.11015). Decomposition & schema-linking; shaped our schema-first prompts and intent taxonomy. • **GPTQ** — E. Frantar et al., ICLR 2023 (arXiv:2210.17323) · **AWQ** — J. Lin et al., MLSys 2024 (arXiv:2306.00978). The 4-bit weight-quantization line of work; our Q4_K_M GGUF applies the same idea to CPU inference. • **CodeS** — G. Qu et al., 2024 (arXiv:2402.16347). Open code-LLMs specialized for SQL; evidence that small code models transfer well to Text-to-SQL.

In one sentence: we hand-taught a small open code model to write SQL for real, messy databases, trained it on free cloud GPUs, and shrank it to under a gigabyte so it answers from any laptop.